

MANIFOLD

A Mathematical Engine for Mapping Vulnerabilities

MANIFOLD Project

Whitepaper v0.1 — working draft, not peer reviewed

Abstract

MANIFOLD treats a software artifact not as a bag of rules to match, but as a *mathematical object*: a typed, weighted graph enriched with *algebraic*, *spectral*, *topological* and *geometric* structure. Over this object we compute a scalar field of **vulnerability potential** $V(x)$, which surfaces anomalous regions as geometric / topological / spectral *features* rather than as literal string matches. An **autonomous LLM agent** navigates the resulting “manifold”, formulates hypotheses about those regions, and dispatches **formal verifiers** (SMT, symbolic execution, abstract interpretation) that confirm or refute each hypothesis — closing the loop between statistical intuition and mathematical proof. This document defines the intermediate representation, formalizes each mathematical layer, states the mapping laws that connect mathematical features to vulnerability classes, and specifies the agent architecture and roadmap.

1 Motivation

Traditional scanners (SAST/DAST) enumerate syntactic patterns. They are brittle: they miss novel vulnerabilities and drown analysts in false positives. Two capabilities are missing:

- (1) A *unified, quantitative representation* of “program structure” onto which many *independent* signals can be projected and fused.
- (2) A *reasoning loop* that converts “suspicious region” into “proven vulnerability” — the LLM supplies the intuition, mathematics supplies the proof.

MANIFOLD provides both.

2 The core idea

artifact $\longrightarrow$ IR (typed graph) $\longrightarrow$ mathematical layers $\longrightarrow$ scalar field $V(x)$

A vulnerability is *not* a pattern; it is a *violation of a structural invariant* (a broken naturality condition, a persistent cycle, a negative curvature chokepoint, a taint flow across a spectral trust cut). Each layer detects a different *kind* of invariant violation, and the fused field $V(x)$ ranks regions for the agent to inspect.

3 The Intermediate Representation (IR)

Definition 1 (Program graph). A *program graph* is a tuple $G = (V, E, \tau_V, \tau_E, \omega)$ where

- V is a finite set of *nodes* (modules, functions, blocks, statements, calls, variables, sources, sinks);
- $E \subseteq V \times V$ is a set of directed *edges*;
- $\tau_V : V \rightarrow \mathbb{N}_K$ assigns each node a *kind* from $K = \{\text{module, function, block, statement, call, assign, variable}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ assigns each edge a *type* $\mathcal{T} = \{\text{control, data, call, taint, trust, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ assigns each edge a non-negative *weight* (default 1).

The IR is the single source of truth. Every adapter (source code, binary, web API, LLM agent) must produce a program graph; every analysis consumes it. The schema is language-agnostic and JSON-serializable (the Python and Rust layers share the identical schema).

4 Mathematical layers

For each layer we fix a subgraph by edge type τ_E , then compute structure.

4.1 Spectral layer (algebraic graph theory)

For an edge type t , define the symmetric *adjacency* A_t and *degree* D_t , and the *combinatorial Laplacian*

$$L_t = D_t - A_t.$$

L_t is positive semidefinite; its spectrum $0 = \lambda_1 \leq \lambda_2 \leq \dots$ encodes global structure:

- λ_2 is the *algebraic connectivity* (Fiedler value). Its eigenvector f_2 — the *Fiedler vector* — induces a canonical cut $S^+ = \{v : f_2(v) \geq 0\}$.
- The *spectral embedding* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ maps nodes into $\mathbb{R}^d$.
- The Cheeger inequality bounds the bottleneck ratio $h(G)$ by $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Signal 1 (spectral anomaly). The reconstruction error of a node under a low-rank eigenmap reconstruction, and its distance from the Fiedler cut, are used as anomaly scores.

4.2 Topological layer (persistent homology)

From the program graph we build a *filtration* of simplicial complexes (e.g. a Vietoris–Rips or flag complex over the shortest-path metric, or a filtration driven by ω). Persistent homology yields, for each dimension k , a *persistence diagram* Dgm_k of birth–death pairs. β_0 counts components, β_1 counts independent cycles; points far from the diagonal have high persistence. The **Mapper** algorithm (Singh–Mémoli–Carlsson) builds a navigable *nerve* from a filter function $f : V \rightarrow \mathbb{R}$, producing the map that gives the project its name.

4.3 Geometric layer (discrete curvature)

The *Ollivier–Ricci curvature* of an edge (u, v) is

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

with lazy random-walk measures m_u, m_v and Wasserstein-1 distance W_1 . Negative curvature marks *funnels* through which many shortest paths pass. The *Forman–Ricci* curvature (unweighted) is $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, cheaply flagging high-degree hubs.

4.4 Algebraic layer (lattices & category theory)

Abstract interpretation. The *taint lattice* $\mathbb{T} = \{\perp = \text{clean} \sqsubseteq \top = \text{tainted}\}$ (a product lattice over tags) gives a *Galois connection* (α, γ) between the concrete collecting semantics and the abstract taint domain, guaranteeing soundness. *Category theory.* The program is a category **Prog**; effects are monads, authorization is a functor $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$. **Design law (naturality of taint):** a vulnerability is a taint flow that breaks the naturality square of F .

4.5 Formal layer (symbolic execution & model checking)

Symbolic execution computes a path condition Φ and symbolic state σ ; an SMT solver checks $\text{SAT}(\Phi \wedge \text{bad})$, whose model is a concrete exploit. Reachability is the CTL formula **EF** bad; taint propagation is a finite automaton whose accepted language is exactly the set of source $\rightarrow$ sink paths.

5 The vulnerability scalar field

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

where s_i are normalized layer signals (spectral anomaly, persistence outlier, curvature, lattice height, formal reachability), α_i are (learnable) weights, and σ a logistic squeeze. V is a ranking, a Mapper filter, and a cost field for navigation.

6 Mapping laws

#	Law	Vulnerability class
L1	Chokepoint ($\kappa \ll 0$)	privilege escalation, auth bypass
L2	Cycle (persistent H_1)	reentrancy, infinite recursion
L3	Trust cut (Fiedler cut)	injection across trust boundary
L4	Naturality break	arbitrary taint violation
L5	Persistence outlier	real vs. spurious features
L6	Reachability ($\text{SAT}(\Phi \wedge \text{bad})$)	concrete exploit

7 The autonomous LLM agent

A closed loop: (1) *ingest* into the IR; (2) *embed & map* ($V(x)$, spectral embedding, persistence, curvature, Mapper); (3) *hypothesize* (the LLM proposes candidates from the manifold); (4) *verify*

(the formal layer proves or refutes); (5) *refine* (findings re-weight and re-embed); (6) *report*. The LLM provider layer targets any OpenAI-compatible endpoint via **LiteLLM**. The LLM does not do raw detection: mathematics proposes regions, the LLM proposes hypotheses, the formal layer proposes proofs.

8 Implementation status

Phase	Deliverable	Status
F0	Whitepaper, architecture	done
F1	IR + Python SAST ingest + taint	done
F2	Spectral + geometric kernels (Rust)	done
F3	TDA (persistent homology, Mapper)	done
F4	Formal verification (Z3, angr)	pending
F5	Autonomous LLM agent (LiteLLM)	pending
F6	Visualization + end-to-end demo	pending
F7	Binary / web / LLM adapters	pending

9 Validation and responsible use

Validation uses toy micro-benchmarks with known ground truth, then labeled corpora (OWASP Benchmark, SARD) once F4 lands, measuring precision/recall of the fused field versus raw taint. MANIFOLD is a defensive / authorized-testing tool; we release under the assumption of authorized use and encourage coordinated disclosure.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.